

01_primary_experiment

February 16, 2026

1 01 — Primary experiment

Purpose: Load all runs for each dataset, perform global Rashomon selection ($K=25$), compute multiplicity and spatial metrics, aggregate across seeds (mean $\pm$ std), and produce summary tables for thesis inclusion.

Uses the experiment runner and analysis module; no training.

1.1 Imports and path setup

```
[5]: import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

import pandas as pd

from analysis.experiment_runner import run_dataset_experiment,
↳run_all_experiments

RESULTS_DIR = ROOT / "results"
```

1.2 Load results

Run the experiment runner for each dataset (or a single dataset). This loads artifacts from `results/{dataset}/seed=*`, performs Rashomon selection, computes multiplicity and spatial metrics, and writes `summary_per_run.csv` per dataset.

```
[6]: # Single dataset (e.g. compas) or all datasets under results/
DATASET = "compas" # set to None to run all datasets

if DATASET:
    dataset_dir = RESULTS_DIR / DATASET
    agg_df = run_dataset_experiment(dataset_dir, verbose=True)
```

```

else:
    agg_df = run_all_experiments(RESULTS_DIR, verbose=True)

agg_df

```

```

Run 1/10: seed=0
Run 2/10: seed=1
Run 3/10: seed=2
Run 4/10: seed=3
Run 5/10: seed=4
Run 6/10: seed=5
Run 7/10: seed=6
Run 8/10: seed=7
Run 9/10: seed=8
Run 10/10: seed=9

```

```

[6]:  dataset  n_runs  mean_variance_mean  mean_variance_std  moran_i_mean  \
0  compas      10           0.001308           0.000306           0.199279

      moran_i_std  n_hh_mean  frac_significant_moran  null_mean  null_std
0           0.077227        80.7                1.0 -0.000433  0.006064

```

1.3 Per-run diagnostics table

The experiment runner writes `results/{dataset}/summary_per_run.csv` when you run the **Load results** cell above. If you run this cell without running that one, we ensure the file exists by running the experiment for that dataset first, then load and display.

```

[7]: def load_per_run_summary(results_dir: Path, dataset: str) -> pd.DataFrame:
    path = results_dir / dataset / "summary_per_run.csv"
    if not path.exists():
        return pd.DataFrame()
    return pd.read_csv(path)

if DATASET:
    per_run = load_per_run_summary(RESULTS_DIR, DATASET)
    if per_run.empty:
        # File not created yet (e.g. ran this cell without "Load results"): run
        ↪ experiment to write it
        run_dataset_experiment(RESULTS_DIR / DATASET, verbose=True)
        per_run = load_per_run_summary(RESULTS_DIR, DATASET)
    display(per_run)
else:
    for d in agg_df["dataset"].unique():
        pr = load_per_run_summary(RESULTS_DIR, d)
        if pr.empty:
            run_dataset_experiment(RESULTS_DIR / d, verbose=False)
            pr = load_per_run_summary(RESULTS_DIR, d)

```

```

if not pr.empty:
    print(f"---- {d} ----")
    display(pr)

```

	outer_seed	mean_variance	moran_i	moran_p_sim	p_empirical	n_hh \
0	0	0.000841	0.140503	0.001	0.009901	48
1	1	0.001524	0.211425	0.001	0.009901	104
2	2	0.000841	0.205800	0.001	0.009901	55
3	3	0.001145	0.360500	0.001	0.009901	160
4	4	0.001416	0.110097	0.001	0.009901	21
5	5	0.001295	0.140545	0.001	0.009901	46
6	6	0.001455	0.175005	0.001	0.009901	102
7	7	0.001725	0.215078	0.001	0.009901	112
8	8	0.001642	0.291669	0.001	0.009901	137
9	9	0.001196	0.142169	0.001	0.009901	22

	null_mean	null_std
0	-0.001206	0.005986
1	-0.001447	0.005477
2	0.000565	0.006508
3	-0.000228	0.005996
4	-0.000695	0.006016
5	-0.000694	0.006660
6	-0.000866	0.006128
7	0.000109	0.006354
8	0.000096	0.005588
9	0.000032	0.005932

1.4 Aggregated dataset table

Final summary: mean $\pm$ std across runs, suitable for thesis tables.

```

[8]: # Format for thesis: e.g. mean_variance as "mean  $\pm$  std"
def format_mean_std(mean_val: float, std_val: float) -> str:
    return f"{mean_val:.4f}  $\pm$  {std_val:.4f}"

summary_display = agg_df.copy()
if "mean_variance_mean" in summary_display.columns:
    summary_display["mean_variance"] = summary_display.apply(
        lambda r: format_mean_std(r["mean_variance_mean"],
        r["mean_variance_std"]), axis=1
    )
if "moran_i_mean" in summary_display.columns:
    summary_display["moran_i"] = summary_display.apply(
        lambda r: format_mean_std(r["moran_i_mean"], r["moran_i_std"]), axis=1
    )

display(summary_display)

```

	dataset	n_runs	mean_variance_mean	mean_variance_std	moran_i_mean	\
0	compas	10	0.001308	0.000306	0.199279	

	moran_i_std	n_hh_mean	frac_significant_moran	null_mean	null_std	\
0	0.077227	80.7	1.0	-0.000433	0.006064	

	mean_variance	moran_i
0	0.0013 ± 0.0003	0.1993 ± 0.0772

1.5 Interpretation

- **mean_variance:** Average predictive variance across test points (Rashomon models). Higher values indicate more multiplicity.
- **moran_i:** Global Moran's I of pointwise variance; positive values indicate spatial clustering of high-variance points.
- **n_hh_mean:** Average number of high-high (HH) LISA hotspots per run.
- **frac_significant_moran:** Fraction of runs where observed Moran's I was significant vs the permutation null ($p_{\text{empirical}} < 0.05$).
- **null_mean / null_std:** Mean and spread of the null distribution of Moran's I (permuted predictions).